WEEK 2 · LESSON 10 OF 24 · HANDOUT

Conversations, Memory and Streaming

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=zjkBMFhNj_g — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: How I use LLMs — https://www.youtube.com/watch?v=EWvNQjAaOHw

🎓 **Go deeper:** Andrej Karpathy: Intro to Large Language Models (1-hour talk) — https://www.youtube.com/watch?v=zjkBMFhNj_g

Your goal in this lesson: Build a multi-turn chatbot, manage the context window by counting and trimming tokens, summarise long histories, and stream answers word by word.

The model has no memory

Each API call is independent: the model remembers **nothing** between calls. A chat "remembers" only because the app sends the **whole conversation so far** every time.

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

A simple chat class

```
class Chat:
    def __init__(self, system, max_history_tokens=2000):
        self.system = system
        self.history = []                       # list of {"role", "content"}
        self.max_history_tokens = max_history_tokens

    def send(self, user_text):
        self.history.append({"role": "user", "content": user_text})
        self.trim()
        response = client.messages.create(model=MODEL, max_tokens=400,
                                          system=self.system, messages=self.history)
        reply = response.content[0].text
        self.history.append({"role": "assistant", "content": reply})
        return reply

    def tokens(self):
        return client.messages.count_tokens(model=MODEL, system=self.system,
                                            messages=self.history).input_tokens

    def trim(self):
        # drop the oldest question-and-answer pair until the history fits
        while len(self.history) > 2 and self.tokens() > self.max_history_tokens:
            self.history = self.history[2:]

bot = Chat("You are a helpful assistant for a Kigali library. Keep answers short.")
print(bot.send("Hello! My name is Aline. Do you have books about farming?"))
print(bot.send("What is my name?"))
print("history messages:", len(bot.history), "| tokens:", bot.tokens())
```

The second answer can use "Aline" only because the first exchange was sent again with the new question.

Why trim?

Every token of history is sent — and paid for — on **every** turn, and the total must fit the model's **context window**. A long chat gets slower and more expensive with each message. `trim()` keeps the most recent turns within a budget, measured with the API's own `count_tokens` (exact for that model, unlike guessing from characters).

The messages list must start with a **user** turn and alternate user/assistant, which is why the trimming removes a whole question-and-answer pair at a time.

Summarise instead of forgetting

Dropping old turns loses information ("My name is Aline"). A better approach for long chats: replace old turns with a short summary.

```
def summarise_history(history):
    transcript = "\n".join(f"{m['role']}: {m['content']}" for m in history)
    return ask("Summarise this conversation in at most 3 short bullet points, keeping names, "
               "numbers and decisions:\n\n" + transcript, max_tokens=200)

def compact(chat, keep_last=2):
    old, recent = chat.history[:-keep_last], chat.history[-keep_last:]
    if not old:
        return
    summary = summarise_history(old)
    chat.system += "\n\nSummary of the earlier conversation:\n" + summary
    chat.history = recent

compact(bot)
print("history messages after compacting:", len(bot.history))
print(bot.system[-300:])
```

The summary moves into the system prompt; the recent turns stay word for word. Many production chatbots combine both: keep the last few turns exactly, summarise the rest.

Long-term memory

For memory across days (a customer's preferences, a student's progress), store facts in your **own database** and add the relevant ones to the prompt when needed. Week 3's retrieval techniques do exactly this for documents. Store only what you need, and tell users what you keep — personal data is protected by law in Rwanda.

Streaming: show words as they arrive

A long answer can take several seconds. **Streaming** sends the text in small pieces as it is generated, so people start reading immediately:

```
with client.messages.stream(
    model=MODEL, max_tokens=300,
    messages=[{"role": "user", "content": "Give three tips for saving money as a student."}],
) as stream:
    for piece in stream.text_stream:
        print(piece, end="", flush=True)
    final = stream.get_final_message()
print("\n\noutput tokens:", final.usage.output_tokens)
```

Same total time, but it **feels** much faster. Web apps forward these pieces to the browser (for example with server-sent events).

Key terms

- **Stateless** — the API keeps no memory between calls.
- **Conversation history** — earlier messages your app re-sends each turn.
- **Context window budget** — a token limit you choose for the history.
- **Summarisation memory** — replacing old turns with a summary.
- **Streaming** — receiving the answer in pieces as it is generated.

Remember this: Chat memory is your app re-sending history; keep it within a token budget by trimming or summarising, store long-term facts yourself, and stream answers so people are not left waiting.

Name: Date:

Activity

Task 1

Set `max_history_tokens=150` and chat for six turns. When does the bot forget your name?

Task 2

Use `compact` instead, and check whether the bot still knows your name.

Task 3

Print `bot.tokens()` after each turn and plot how it grows.

Task 4

Write a `Chat` method `reset()` that starts a new conversation but keeps the system prompt.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. How does a chatbot 'remember' earlier messages?
   - A. The API key stores them
   - B. The app re-sends the conversation history each turn
   - C. The model stores them
   - D. It doesn't remember anything
2. Why trim or summarise long conversations?
   - A. To change the system prompt
   - B. History costs tokens every turn and must fit the context window
   - C. Old messages are illegal
   - D. The model gets angry
3. What is the advantage of summarising over simply dropping old turns?
   - A. It is free
   - B. It makes the model bigger
   - C. It removes the system prompt
   - D. Important facts such as names and decisions are kept
4. Why remove a whole question-and-answer pair when trimming?
   - A. The API deletes them anyway
   - B. It is random
   - C. Pairs are cheaper
   - D. Messages must start with the user and alternate roles
5. What does streaming improve?
   - A. The context window
   - B. Accuracy
   - C. The total cost
   - D. How fast the answer feels, by showing words as they arrive
6. Where should long-term memory such as user preferences be kept?
   - A. Nowhere
   - B. Inside the model
   - C. In the API key
   - D. In your own database, added to prompts when needed

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.